

VLLM专题（三十五）—多模态数据处理



大模型专题系列 专栏收录该内容

您已是超级会员，正在免费阅读会员专享内容

[查看更多超级会员权益](#)

为了在vLLM中实现各种优化，例如分块预填充和前缀缓存，我们使用BaseMultiModalProcessor来提供占位符特征标记（例如）与多模态输入（例如原始输入图像）之间的对应关系，基于HF处理器的输出。

以下是BaseMultiModalProcessor的主要特性：

1. 提示更新检测

HF处理器的主要职责之一是使用占位符标记更新提示。例如：

在字符串的开头插入特征占位符标记（例如...，其数量等于特征大小）。

用特征占位符标记（例如...，其数量等于特征大小）替换现有的输入占位符标记（例如表示单个图像）。

关于哪些标记已被更新的信息是找到占位符特征标记与多模态输入之间对应关系的关键。

在vLLM中，此信息通过_get_prompt_updates()中的PromptUpdate指定。我们可以通过检查更新后的标记是否存在来自动检测HF是否已更新提示。

2. 分词后的提示输入

为了支持在单独进程中进行分词，我们允许将输入的分词ID与多模态数据一起传递。

2.1 问题

考虑到HF处理器通常遵循以下主要步骤：

1. 对文本进行分词
2. 处理多模态输入
3. 执行提示更新

而我们的需求是：

- 对于**文本 + 多模态输入**，应用所有步骤1–3。
- 对于**分词后的输入 + 多模态输入**，仅应用步骤2–3。

如何在不重写HF处理器的情况下实现这一点？我们可以尝试在不同输入上多次调用HF处理器：

- 对于**文本 + 多模态输入**，直接调用HF处理器即可。
- 对于**分词后的输入 + 多模态输入**，仅对多模态输入调用处理器。

然而，HF处理器原生支持**文本 + 多模态输入**，但对于**分词后的输入 + 多模态输入**则不然：如果输入占位符标记的数量与多模态输入的数量不匹配，则会抛出错误。

此外，由于分词后的文本未通过HF处理器处理，我们必须自行执行步骤3，以确保输出的分词与多模态数据保持一致。

2.2 虚拟文本

我们通过要求每个模型定义如何根据多模态输入的数量生成虚拟文本来解决第一个问题，具体通过 `get_dummy_processor_inputs()` 实现。这使得我们能够生成与多模态输入对应的虚拟文本，并将其一起输入以获取处理后的多模态数据。

2.3 自动提示更新

我们通过在 `_apply_prompt_updates()` 中实现与模型无关的代码来解决第二个问题，该代码会根据 ```_get_prompt_updates()``` 输出的规范自动使用特征占位符标记更新提示。

2.4 总结

借助虚拟文本和自动提示更新，我们的多模态处理器最终能够接受带有文本或分词提示的多模态数据。详细逻辑如 `_apply_hf_processor_main()` 所示。

3. 处理器输出缓存

一些HF处理器（例如Qwen2-VL的处理器）运行速度非常慢。为了缓解这个问题，我们缓存了HF处理器的多模态输出，以避免重复处理相同的多模态输入（例如图像）。

当新数据传入时，我们首先检查哪些数据项已经在缓存中，哪些缺失。缺失的数据项会被批量传递给HF处理器进行处理并缓存，然后与缓存中已有的数据项合并。

由于我们只处理缺失的多模态数据项，输入占位符标记的数量不再与多模态输入的数量对应，因此它们不能与文本提示一起传递给HF处理器。因此，我们使用虚拟文本来分别处理文本和多模态输入，以避免HF错误。由于跳过了HF的提示更新代码，我们在之后应用自动提示更新，以确保输出的分词与多模态数据保持一致。